

=====

CAPSTONE PROJECT

NOTEBOOK 43: FULL-161 BATCH INFERENCE DEMO

=====

Purpose:

This notebook applies the final full-161 pipeline to a batch

of test tracks so the project has a multi-file deployment demo.

The goal is to:

1. Load the frozen Stage-1 benchmark setup
2. Load the Stage-2 rare-tail router
3. Run the full-161 pipeline on a sample of expanded test tracks
4. Record candidate predictions and rare-tail suggestions

5. Compare predictions with available ground truth

6. Save report-ready demo tables

=====

In [1]:

```
# =====
# 1. IMPORT LIBRARIES
# =====

import os
import json
import warnings
from pathlib import Path

import joblib
import librosa
import numpy as np
import pandas as pd
import tensorflow as tf

from scipy.stats import skew, kurtosis

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning)

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("Seed set to:", SEED)
print("TensorFlow version:", tf.__version__)
```

Seed set to: 42
TensorFlow version: 2.20.0

In [2]:

```
# =====
# 2. LOAD FROZEN ARTIFACTS
# =====

structured_model =
joblib.load("../models/final_structured_multilabel_candidate150_best_model.joblib")
structured_scaler =
joblib.load("../models/final_structured_multilabel_candidate150_scaler.joblib")
```

```

audio_model =
tf.keras.models.load_model("../models/audio_multilabel_candidate150_expanded_final.
keras")

candidate_label_cols = np.load(
    "../data/processed/hybrid_multilabel_candidate150_expanded_label_columns.npy",
    allow_pickle=True
)

stage1_config = {}
with
open("../data/processed/hybrid_multilabel_candidate150_expanded_best_config.txt",
"r") as f:
    for line in f:
        line = line.strip()
        if "=" in line:
            k, v = line.split("=")
            stage1_config[k.strip()] = float(v.strip())

STAGE1_STRUCTURED_WEIGHT = stage1_config["structured_weight"]
STAGE1_AUDIO_WEIGHT = stage1_config["audio_weight"]
STAGE1_THRESHOLD = stage1_config["threshold"]

rare_tail_router_df =
pd.read_csv("../data/processed/full161_rare_tail_routing_table.csv")
genre_inventory_df = pd.read_csv("../data/processed/full_genre_inventory.csv")
full_master_df = pd.read_csv("../data/processed/multilabel_full_master_table.csv")

stage2_config = {}
with open("../data/processed/full161_stage2_best_config.txt", "r") as f:
    for line in f:
        line = line.strip()
        if "=" in line:
            k, v = line.split("=")
            stage2_config[k.strip()] = v.strip()

STAGE2_ANCHOR_TRIGGER_THRESHOLD = float(stage2_config["anchor_trigger_threshold"])
STAGE2_TOP_K = int(stage2_config["top_k"])

features_reference = pd.read_csv(
    "../data/raw/metadata/features.csv",
    header=[0, 1, 2],
    index_col=0
)

expanded_test_df =
pd.read_csv("../data/processed/audio_multilabel_candidate150_expanded_test.csv")

print("Structured model loaded.")
print("Audio model loaded.")
print("Candidate labels:", len(candidate_label_cols))
print("Stage-1 config:", stage1_config)
print("Stage-2 config:", stage2_config)
print("Rare-tail router shape:", rare_tail_router_df.shape)
print("Expanded test shape:", expanded_test_df.shape)
print("Reference features shape:", features_reference.shape)

```

```

Structured model loaded.
Audio model loaded.
Candidate labels: 150
Stage-1 config: {'structured_weight': 0.1, 'audio_weight': 0.9, 'threshold': 0.2}
Stage-2 config: {'anchor_trigger_threshold': '0.1', 'top_k': '1'}
Rare-tail router shape: (13, 26)
Expanded test shape: (1500, 158)
Reference features shape: (106574, 518)

```

In [3]:

```

# =====
# 3. PREPARE LOOKUPS
# =====

genre_inventory_df["genre_id"] = genre_inventory_df["genre_id"].astype(int)
genre_name_map = dict(zip(genre_inventory_df["genre_id"],
                           genre_inventory_df["genre_name"]))

candidate_label_ids = [int(col.replace("genre_", "")) for col in
                        candidate_label_cols]
candidate_id_to_index = {int(col.replace("genre_", "")): i for i, col in
                          enumerate(candidate_label_cols)}

fallback_router_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Hierarchy-triggered fallback"
].copy().reset_index(drop=True)

fallback_router_df["rare_tail_genre_id"] =
fallback_router_df["rare_tail_genre_id"].astype(int)
fallback_router_df["anchor_candidate_id"] =
fallback_router_df["anchor_candidate_id"].astype(int)

inventory_only_df = rare_tail_router_df[
    rare_tail_router_df["fallback_mode"] == "Inventory only"
].copy().reset_index(drop=True)

features_reference.columns = [
    "_".join([str(level) for level in col]).strip()
    for col in features_reference.columns.to_flat_index()
]

reference_feature_df = features_reference.copy()
reference_feature_df = reference_feature_df.select_dtypes(include=["number"])
reference_feature_df = reference_feature_df.replace([np.inf, -np.inf], np.nan)
reference_feature_means = reference_feature_df.mean(axis=0)
reference_feature_columns = list(reference_feature_df.columns)

full_master_indexed = full_master_df.set_index("track_id", drop=False)

print("Fallback rare-tail labels:", fallback_router_df.shape[0])
print("Inventory-only rare-tail labels:", inventory_only_df.shape[0])
print("Structured reference feature columns:", len(reference_feature_columns))

```

Fallback rare-tail labels: 10
Inventory-only rare-tail labels: 3
Structured reference feature columns: 518

In [4]:

```
# =====  
# 4. DEMO SETTINGS  
# =====  
  
SR = 22050  
DURATION = 15  
N_MELS = 64  
N_FFT = 2048  
HOP_LENGTH = 1024  
MAX_FRAMES = int(np.ceil((DURATION * SR) / HOP_LENGTH)) + 1  
  
# Number of demo tracks to sample from expanded test  
N_DEMO_TRACKS = 30  
  
# Reproducible sampling  
demo_df = expanded_test_df.sample(n=N_DEMO_TRACKS,  
random_state=SEED).reset_index(drop=True)  
  
print("SR:", SR)  
print("DURATION:", DURATION)  
print("N_MELS:", N_MELS)  
print("MAX_FRAMES:", MAX_FRAMES)  
print("Demo sample shape:", demo_df.shape)  
  
display(demo_df.head(10))
```

SR: 22050
DURATION: 15
N_MELS: 64
MAX_FRAMES: 324
Demo sample shape: (30, 158)

	track_id	split	subset	genre_top	title	audio_path	a
0	118838	test	large	NaN	XXXV	../data/raw/audio/fma_large\118\118838.mp3	
1	142325	test	large	NaN	Blue Sky	../data/raw/audio/fma_large\142\142325.mp3	
2	48540	test	large	NaN	Introduction from Mike	../data/raw/audio/fma_large\048\048540.mp3	
3	48174	test	large	NaN	Simple Elegance	../data/raw/audio/fma_large\048\048174.mp3	
4	51231	test	large	NaN	Is She Secretly On My Side? (Soundtrack of Her...	../data/raw/audio/fma_large\051\051231.mp3	
5	96091	test	large	NaN	The Big Eleven	../data/raw/audio/fma_large\096\096091.mp3	
6	115656	test	large	NaN	My Paris is Here	../data/raw/audio/fma_large\115\115656.mp3	
7	84465	test	large	NaN	Home Sweet Home [recycled by Sektor 304]	../data/raw/audio/fma_large\084\084465.mp3	
8	133220	test	large	NaN	Frozen Bong	../data/raw/audio/fma_large\133\133220.mp3	
9	33045	test	large	Rock	Later Days	../data/raw/audio/fma_large\033\033045.mp3	

10 rows × 158 columns

In [5]:

```
# =====
# 5. HELPER FUNCTIONS
# =====

def load_audio(file_path, sr=SR, duration=DURATION):
    y, sr_loaded = librosa.load(file_path, sr=sr, mono=True, duration=duration)
    if y is None or len(y) == 0:
        raise ValueError(f"Could not load usable audio from: {file_path}")
    return y, sr_loaded

def build_mel_input(y, sr=SR, n_mels=N_MELS, n_fft=N_FFT, hop_length=HOP_LENGTH,
max_frames=MAX_FRAMES):
    mel = librosa.feature.melspectrogram(
        y=y,
        sr=sr,
        n_fft=n_fft,
        hop_length=hop_length,
        n_mels=n_mels
```

```

)
mel_db = librosa.power_to_db(mel, ref=np.max)
mel_db = np.clip((mel_db + 80.0) / 80.0, 0.0, 1.0)

if mel_db.shape[1] < max_frames:
    pad_width = max_frames - mel_db.shape[1]
    mel_db = np.pad(mel_db, ((0, 0), (0, pad_width)), mode="constant")
else:
    mel_db = mel_db[:, :max_frames]

return mel_db.astype(np.float32)[None, :, :, None]

def safe_stat_vector(arr_2d, stat_name):
    arr_2d = np.asarray(arr_2d, dtype=np.float64)

    if arr_2d.ndim == 1:
        arr_2d = arr_2d.reshape(1, -1)

    if stat_name == "mean":
        out = np.mean(arr_2d, axis=1)
    elif stat_name == "std":
        out = np.std(arr_2d, axis=1)
    elif stat_name == "median":
        out = np.median(arr_2d, axis=1)
    elif stat_name == "min":
        out = np.min(arr_2d, axis=1)
    elif stat_name == "max":
        out = np.max(arr_2d, axis=1)
    elif stat_name == "skew":
        out = skew(arr_2d, axis=1, bias=False, nan_policy="omit")
    elif stat_name == "kurtosis":
        out = kurtosis(arr_2d, axis=1, bias=False, nan_policy="omit")
    else:
        raise ValueError(f"Unknown stat: {stat_name}")

    out = np.asarray(out, dtype=np.float64)
    out[~np.isfinite(out)] = 0.0
    return out.astype(np.float32)

def build_feature_matrices(y, sr=SR):
    y = np.asarray(y, dtype=np.float64)
    y_harmonic = librosa.effects.harmonic(y)

    mats = {}
    mats["chroma_stft"] = librosa.feature.chroma_stft(y=y, sr=sr)
    mats["chroma_cqt"] = librosa.feature.chroma_cqt(y=y, sr=sr)
    mats["chroma_cens"] = librosa.feature.chroma_cens(y=y, sr=sr)
    mats["tonnetz"] = librosa.feature.tonnetz(y=y_harmonic, sr=sr)
    mats["mfcc"] = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=20)
    mats["rms"] = librosa.feature.rms(y=y)
    mats["spectral_centroid"] = librosa.feature.spectral_centroid(y=y, sr=sr)
    mats["spectral_bandwidth"] = librosa.feature.spectral_bandwidth(y=y, sr=sr)
    mats["spectral_contrast"] = librosa.feature.spectral_contrast(y=y, sr=sr)
    mats["spectral_rolloff"] = librosa.feature.spectral_rolloff(y=y, sr=sr)
    mats["zcr"] = librosa.feature.zero_crossing_rate(y)

    return mats

```

```

def build_structured_feature_vector(y, reference_columns, reference_means, sr=SR):
    feature_mats = build_feature_matrices(y, sr=sr)
    row_dict = {}

    for col in reference_columns:
        parts = col.split("_")
        component_idx = int(parts[-1]) - 1
        stat_name = parts[-2]
        feature_name = "_".join(parts[:-2])

        if feature_name in feature_mats:
            mat = feature_mats[feature_name]
            stat_vec = safe_stat_vector(mat, stat_name)

            if 0 <= component_idx < len(stat_vec):
                row_dict[col] = float(stat_vec[component_idx])
            else:
                row_dict[col] = np.nan
        else:
            row_dict[col] = np.nan

    X_one = pd.DataFrame([row_dict], columns=reference_columns)
    X_one = X_one.replace([np.inf, -np.inf], np.nan)

    for col in reference_columns:
        if pd.isna(X_one.loc[0, col]):
            X_one.loc[0, col] = float(reference_means[col])

    return X_one.astype(np.float32)

def scores_to_pseudoprobs(score_matrix):
    clipped = np.clip(score_matrix, -20, 20)
    return 1.0 / (1.0 + np.exp(-clipped))

def get_structured_scores(model, X_scaled):
    if hasattr(model, "predict_proba"):
        scores = model.predict_proba(X_scaled)
    elif hasattr(model, "decision_function"):
        scores = model.decision_function(X_scaled)
    else:
        raise ValueError("Structured model supports neither predict_proba nor decision_function.")
    return np.asarray(scores)

def fuse_probabilities(structured_probs, audio_probs, w_structured, w_audio):
    return (w_structured * structured_probs) + (w_audio * audio_probs)

def decode_stage1(prob_matrix, threshold):
    return (prob_matrix >= threshold).astype(np.uint8)

def build_rare_tail_scores_for_single(stage1_probs, router_df,
candidate_index_map):
    scores = []

    for _, row in router_df.iterrows():
        anchor_id = int(row["anchor_candidate_id"])

```



```

        anchor_idx = candidate_index_map[anchor_id]
        anchor_prob = float(stage1_probs[0, anchor_idx])

        p_anchor = 0.0 if pd.isna(row["p_rare_given_anchor"]) else
float(row["p_rare_given_anchor"])
        p_root = 0.0 if pd.isna(row["p_rare_given_root"]) else
float(row["p_rare_given_root"])

        strength = max(p_anchor, p_root)
        rare_score = anchor_prob * strength

        scores.append({
            "rare_tail_genre_id": int(row["rare_tail_genre_id"]),
            "rare_tail_genre_name": row["rare_tail_genre_name"],
            "anchor_candidate_id": anchor_id,
            "anchor_candidate_name": row["anchor_candidate_name"],
            "anchor_prob": anchor_prob,
            "rare_tail_score": rare_score,
            "fallback_mode": row["fallback_mode"]
        })

    return pd.DataFrame(scores)

def get_true_labels(track_id):
    row = full_master_indexed.loc[track_id]

    true_candidate_ids = []
    for col in candidate_label_cols:
        if int(row[col]) == 1:
            true_candidate_ids.append(int(col.replace("genre_", "")))

    fallback_cols = [f"genre_{gid}" for gid in
fallback_router_df["rare_tail_genre_id"].astype(int).tolist()]
    true_rare_ids = []
    for col in fallback_cols:
        if col in row.index and int(row[col]) == 1:
            true_rare_ids.append(int(col.replace("genre_", "")))

    return (
        true_candidate_ids,
        [genre_name_map.get(gid, str(gid)) for gid in true_candidate_ids],
        true_rare_ids,
        [genre_name_map.get(gid, str(gid)) for gid in true_rare_ids],
    )

```

In [6]:

```

# =====
# 6. RUN FULL-161 PIPELINE ACROSS DEMO TRACKS
# =====

batch_rows = []
problem_files = []

for _, row in demo_df.iterrows():

```

```

track_id = int(row["track_id"])
audio_path = row["audio_path"]

try:
    y, sr_loaded = load_audio(audio_path, sr=SR, duration=DURATION)

    X_audio_input = build_mel_input(
        y,
        sr=sr_loaded,
        n_mels=N_MELS,
        n_fft=N_FFT,
        hop_length=HOP_LENGTH,
        max_frames=MAX_FRAMES
    )

    X_structured_one = build_structured_feature_vector(
        y,
        reference_feature_columns,
        reference_feature_means,
        sr=sr_loaded
    )

    X_structured_scaled =
structured_scaler.transform(X_structured_one).astype(np.float32)

    structured_scores = get_structured_scores(structured_model,
X_structured_scaled)
    structured_probs = scores_to_pseudoprobs(structured_scores)

    audio_probs = audio_model.predict(X_audio_input, verbose=0)

    stage1_probs = fuse_probabilities(
        structured_probs,
        audio_probs,
        STAGE1_STRUCTURED_WEIGHT,
        STAGE1_AUDIO_WEIGHT
    )

    stage1_pred = decode_stage1(stage1_probs, STAGE1_THRESHOLD)

    predicted_candidate_ids = [
        int(col.replace("genre_", ""))
        for j, col in enumerate(candidate_label_cols)
        if int(stage1_pred[0, j]) == 1
    ]
    predicted_candidate_names = [genre_name_map.get(gid, str(gid)) for gid in
predicted_candidate_ids]

    rare_tail_scores_df = build_rare_tail_scores_for_single(
        stage1_probs,
        fallback_router_df,
        candidate_id_to_index
    ).sort_values(["rare_tail_score", "anchor_prob"],
ascending=False).reset_index(drop=True)

    stage2_suggestions_df = rare_tail_scores_df[
        rare_tail_scores_df["anchor_prob"] >= STAGE2_ANCHOR_TRIGGER_THRESHOLD

```

```

].head(STAGE2_TOP_K).copy()

rare_tail_ids =
stage2_suggestions_df["rare_tail_genre_id"].astype(int).tolist() if
len(stage2_suggestions_df) > 0 else []
rare_tail_names = stage2_suggestions_df["rare_tail_genre_name"].tolist() if
len(stage2_suggestions_df) > 0 else []
rare_tail_scores =
stage2_suggestions_df["rare_tail_score"].round(6).tolist() if
len(stage2_suggestions_df) > 0 else []

true_candidate_ids, true_candidate_names, true_rare_ids, true_rare_names =
get_true_labels(track_id)

candidate_hit = any(gid in predicted_candidate_ids for gid in
true_candidate_ids) if len(true_candidate_ids) > 0 else False
rare_tail_hit = any(gid in rare_tail_ids for gid in true_rare_ids) if
len(true_rare_ids) > 0 else False

batch_rows.append({
    "track_id": track_id,
    "audio_path": audio_path,
    "true_candidate_label_count": len(true_candidate_ids),
    "true_candidate_label_names": true_candidate_names,
    "predicted_candidate_label_count": len(predicted_candidate_ids),
    "predicted_candidate_label_names": predicted_candidate_names,
    "candidate_hit_any": candidate_hit,
    "true_rare_tail_label_count": len(true_rare_ids),
    "true_rare_tail_label_names": true_rare_names,
    "stage2_rare_tail_suggestion_count": len(rare_tail_ids),
    "stage2_rare_tail_suggestion_names": rare_tail_names,
    "stage2_rare_tail_scores": rare_tail_scores,
    "rare_tail_hit_any": rare_tail_hit
})

except Exception as e:
    problem_files.append({
        "track_id": track_id,
        "audio_path": audio_path,
        "error": str(e)
    })

batch_results_df = pd.DataFrame(batch_rows)

print("Batch inference results shape:", batch_results_df.shape)
print("Problem files:", len(problem_files))

display(batch_results_df.head(20))

```

Batch inference results shape: (30, 13)

Problem files: 0

	track_id	audio_path	true_candidate_label_count	true_candid
0	118838	../data/raw/audio/fma_large\118\118838.mp3	4	[Experimental]
1	142325	../data/raw/audio/fma_large\142\142325.mp3	4	[Alternative]
2	48540	../data/raw/audio/fma_large\048\048540.mp3	6	[Rock, Hardcore]
3	48174	../data/raw/audio/fma_large\048\048174.mp3	3	[Jazz]
4	51231	../data/raw/audio/fma_large\051\051231.mp3	5	[Pop, Rock]
5	96091	../data/raw/audio/fma_large\096\096091.mp3	3	[Hip-Hop, Rap]
6	115656	../data/raw/audio/fma_large\115\115656.mp3	2	[Classical]
7	84465	../data/raw/audio/fma_large\084\084465.mp3	3	[Rock, Electronic]
8	133220	../data/raw/audio/fma_large\133\133220.mp3	3	[Electronic]
9	33045	../data/raw/audio/fma_large\033\033045.mp3	2	[Jazz]
10	98307	../data/raw/audio/fma_large\098\098307.mp3	3	[International]
11	148770	../data/raw/audio/fma_large\148\148770.mp3	2	[Jazz]
12	80913	../data/raw/audio/fma_large\080\080913.mp3	2	[Electronic]
13	143595	../data/raw/audio/fma_large\143\143595.mp3	3	[Jazz, Electronic]
14	10717	../data/raw/audio/fma_large\010\010717.mp3	4	[Avant-Garde, Experimental]
15	132882	../data/raw/audio/fma_large\132\132882.mp3	4	[Rock, Electronic]
16	154691	../data/raw/audio/fma_large\154\154691.mp3	2	
17	66278	../data/raw/audio/fma_large\066\066278.mp3	3	[Pop, Electronic]
18	49145	../data/raw/audio/fma_large\049\049145.mp3	3	[Pop, Rock, Electronic]
19	9265	../data/raw/audio/fma_large\009\009265.mp3	4	[Pop, Folk, Electronic]



In [7]:

```

# =====
# 7. BUILD DEMO SUMMARY METRICS
# =====

summary_rows = []

candidate_eligible = int((batch_results_df["true_candidate_label_count"] >
0).sum())
candidate_hits = int(batch_results_df["candidate_hit_any"].sum())

rare_tail_eligible = int((batch_results_df["true_rare_tail_label_count"] >
0).sum())
rare_tail_hits = int(batch_results_df["rare_tail_hit_any"].sum())

summary_rows.append({
    "Metric": "Tracks in demo batch",
    "Value": len(batch_results_df)
})

summary_rows.append({
    "Metric": "Candidate-eligible tracks",
    "Value": candidate_eligible
})

summary_rows.append({
    "Metric": "Candidate hit-any count",
    "Value": candidate_hits
})

summary_rows.append({
    "Metric": "Candidate hit-any rate",
    "Value": (candidate_hits / candidate_eligible) if candidate_eligible > 0 else
np.nan
})

summary_rows.append({
    "Metric": "Rare-tail-eligible tracks",
    "Value": rare_tail_eligible
})

summary_rows.append({
    "Metric": "Rare-tail hit-any count",
    "Value": rare_tail_hits
})

summary_rows.append({
    "Metric": "Rare-tail hit-any rate",
    "Value": (rare_tail_hits / rare_tail_eligible) if rare_tail_eligible > 0 else
np.nan
})

summary_rows.append({
    "Metric": "Average predicted candidate labels",
    "Value": float(batch_results_df["predicted_candidate_label_count"].mean()) if

```

```
len(batch_results_df) > 0 else np.nan
}))

summary_rows.append({
    "Metric": "Average rare-tail suggestions",
    "Value": float(batch_results_df["stage2_rare_tail_suggestion_count"].mean()) if
len(batch_results_df) > 0 else np.nan
}))

batch_summary_df = pd.DataFrame(summary_rows)

print("Batch demo summary:")
display(batch_summary_df)
```

Batch demo summary:

	Metric	Value
0	Tracks in demo batch	30.000000
1	Candidate-eligible tracks	30.000000
2	Candidate hit-any count	27.000000
3	Candidate hit-any rate	0.900000
4	Rare-tail-eligible tracks	0.000000
5	Rare-tail hit-any count	0.000000
6	Rare-tail hit-any rate	NaN
7	Average predicted candidate labels	4.333333
8	Average rare-tail suggestions	0.466667

In [8]:

```
# =====
# 8. SHOW TRACKS WITH TRUE RARE-TAIL LABELS
# =====

rare_tail_demo_df = batch_results_df[
    batch_results_df["true_rare_tail_label_count"] > 0
].copy().reset_index(drop=True)

print("Tracks with true rare-tail labels in the demo sample:")
display(rare_tail_demo_df)
```

Tracks with true rare-tail labels in the demo sample:

track_id	audio_path	true_candidate_label_count	true_candidate_label_names	predicted_candi

In [9]:

```

# =====
# 9. SAVE OUTPUTS
# =====

os.makedirs("../data/processed", exist_ok=True)

batch_results_df.to_csv(
    "../data/processed/full161_batch_inference_results.csv",
    index=False
)

batch_summary_df.to_csv(
    "../data/processed/full161_batch_inference_summary.csv",
    index=False
)

rare_tail_demo_df.to_csv(
    "../data/processed/full161_batch_inference_rare_tail_subset.csv",
    index=False
)

if len(problem_files) > 0:
    pd.DataFrame(problem_files).to_csv(
        "../data/processed/full161_batch_inference_problem_files.csv",
        index=False
    )

print("Saved full-161 batch inference demo outputs.")

```

Saved full-161 batch inference demo outputs.

In [10]:

```

# =====
# 10. INTERPRETATION NOTES
# =====

print("1. This notebook demonstrates the final full-161 pipeline across multiple files.")
print("2. Stage 1 produces direct candidate-label predictions for each file.")
print("3. Stage 2 produces rare-tail fallback suggestions only when anchor conditions are met.")
print("4. The exported tables can be used as qualitative deployment evidence in the report or presentation.")
print("5. This notebook is a demo notebook, not a replacement for the formal benchmark notebooks.")

```

1. This notebook demonstrates the final full-161 pipeline across multiple files.
2. Stage 1 produces direct candidate-label predictions for each file.
3. Stage 2 produces rare-tail fallback suggestions only when anchor conditions are met.
4. The exported tables can be used as qualitative deployment evidence in the report or presentation.
5. This notebook is a demo notebook, not a replacement for the formal benchmark notebooks.